

Paradigmes de programmation

Programmation fonctionnelle

1. Qu'est-ce qu'un paradigme ?

Un **paradigme de programmation** est une manière d'appréhender un problème et d'organiser un programme informatique visant à le résoudre. Un même problème peut souvent être résolu selon plusieurs approches, plusieurs paradigmes — et un même langage (comme Python) permet d'en combiner plusieurs au sein d'un même programme.

Les trois paradigmes à connaître en NSI :

Paradigme	Description	Principe directeur
Impératif	Suite d'instructions qui modifient l'état du programme	<i>Comment faire ?</i>
Objet (POO)	Regroupement de données et de comportements dans des objets	<i>Qui fait quoi ?</i>
Fonctionnel	Composition de fonctions pures sans modifier l'état	<i>Que calculer ?</i>

L'**état du programme**, c'est l'ensemble des valeurs stockées en mémoire à un instant donné — variables, listes, compteurs...

- En paradigme impératif, le programme avance en modifiant cet état pas à pas (une boucle incrémente un compteur, une fonction modifie une liste...).
- En POO, l'état est aussi modifié — mais la modification est *encapsulée* : chaque objet gère son propre état, et les autres objets n'y accèdent (idéalement) que *via* des méthodes définies. C'est une forme de contrôle, pas une suppression de l'état.
- En paradigme fonctionnel, l'état **ne change jamais** : on *ne modifie pas* les données existantes, on en *produit de nouvelles*.

Voir les fiches consacrées à Python et à la POO pour les paradigmes impératifs et objets. Le principe fonctionnel est décrit ci-dessous.

2. Le paradigme fonctionnel

2.1. Idée centrale

En programmation fonctionnelle, un programme est une **composition de fonctions** au sens mathématique : chaque fonction prend des données en entrée et produit un résultat en sortie, **sans modifier quoi que ce soit d'autre**.

Remarque : notation mathématique ▼

La *composition* est une opération naturelle en mathématique que, tel M. Jourdain qui faisait de la prose sans le savoir, vous avez employée à de nombreuses reprises sans vous en apercevoir. Ainsi, la fonction f définie par $f(x) = (x + 1)^2$ est en réalité la composée des fonctions a et c respectivement définies par $a(x) = x + 1$ et $c(t) = t^2$.

On a $f(x) = c(a(x))$ car $c(a(x)) = c(x + 1) = (x + 1)^2$. On note alors $f = c \circ a$, donc $f(x) = c \circ a(x)$.

On distingue deux propriétés fondamentales :

- **Fonction *pure*** : pour une même entrée, une fonction *pure* renvoie *toujours* le même résultat et ne produit aucun **effet de bord** (pas de modification de variable globale, pas d'écriture sur disque, pas d'affichage...).
- **Immuabilité** : les données ne sont jamais modifiées *en place* — on en crée une nouvelle version transformée.

Remarque : effet de bord ? ▼

Une fonction a un **effet de bord** lorsqu'elle modifie quelque chose en dehors de sa propre portée : variable globale, liste passée en argument, affichage console, écriture dans un fichier...

- **Avec effet de bord** : la liste originale (passée en argument) est modifiée.

```
def doubler_imperatif(lst):  
    for i in range(len(lst)):  
        lst[i] *= 2          # ← modification en place !
```

- **Sans effet de bord** : la fonction renvoie une nouvelle liste.

```
def doubler_fonctionnel(lst):  
    return [x * 2 for x in lst]    # ← lst est inchangée
```

2.2. Comparaison impératif / fonctionnel

Illustration : on veut calculer la somme des entiers de 1 à n .

- **Paradigme impératif** : on accumule dans une variable.

```
def somme_imperatif(n):  
    total = 0          # état modifiable  
    for i in range(1, n+1):  
        total += i     # modification de l'état  
    return total
```

Remarque : mutation vs réassignation ? ▼

Ce sont deux formes distinctes de modification de l'état.

- **Réassignation** : on relie un nom de variable à un *nouvel* objet. `total += i` (sur un entier) est un raccourci pour `total = total + i` : un *nouvel entier* est créé, et `total` est redirigé pour pointer dessus. *Idem* pour `i` dans `for i in range(...)`. Les entiers Python étant **immuables**, *aucun objet* n'est modifié *en place*.
- **Mutation** : on modifie *l'objet lui-même* en mémoire. Si `lst` est une liste d'entiers, `lst.append(val)` ou `lst[0] *= 2` modifient la liste existante sans en créer une nouvelle. C'est ce que le paradigme fonctionnel interdit explicitement.

En paradigme fonctionnel *strict*, **les deux sont bannis** — mais en Python, c'est surtout la **mutation** qui produit des effets de bord visibles depuis l'extérieur.

- **Paradigme fonctionnel** : on décompose le calcul de la somme récursivement.

```
def somme_fonctionnel(n):  
    if n == 0:  
        return 0  
    return n + somme_fonctionnel(n - 1) # pas de variable modifiée
```

⚠ Récursivité et paradigme fonctionnel

Sans boucles ni affectations, le paradigme fonctionnel s'appuie naturellement sur la **récursivité** pour exprimer les répétitions. Les deux notions sont donc fortement liées.

2.3. map et filter en Python

Python propose nativement deux fonctions d'ordre supérieur (*higher-order functions* — des fonctions qui prennent en paramètre une *autre fonction*) : `map` et `filter`. Elles sont très utilisées dans le style fonctionnel :

- `map(f, lst)` : applique la fonction `f` à chaque élément de `lst`.
- `filter(pred, lst)` : ne conserve que les éléments de `lst` pour lesquels `pred` renvoie une valeur assimilable à `True`.

```
nombres = [1, 2, 3, 4, 5, 6]  
carres = list(map(lambda x: x**2, nombres))      # carres désigne [1, 4, 9, 16, 25, 36]  
pairs = list(filter(lambda x: x % 2 == 0, nombres)) # pairs désigne [2, 4, 6]
```

💬 Remarque : lambda ? ▼

`lambda x: expression` est une **fonction anonyme** (sans nom) — une façon concise de définir une fonction à usage unique, « jetable », sans recourir à `def` ni devoir lui choisir un nom. On utilise cet idiome lorsqu'on ne prévoit pas de réutiliser la fonction en question (un peu comme on peut écrire `for _ in range(...)` lorsque le nom de la variable de boucle importe peu).

`lambda x: x**2` est strictement équivalent à :

```
def carre(x):  
    return x**2
```

Les **compréhensions de listes** Python produisent le même résultat et sont souvent préférées pour leur lisibilité :

```
carres = [x**2 for x in nombres]  
pairs = [x for x in nombres if x % 2 == 0]
```

2.4. reduce depuis functools

La fonction `reduce` vient compléter `map` et `filter` pour former un « trio classique ». Elle n'est pas native en Python : il faut l'importer depuis le module `functools`, consacré aux fonctions d'ordre supérieur.

`reduce(f, lst)` **combine** les éléments de `lst` (une liste Python) en une valeur unique, en appliquant `f` de gauche à droite. La fonction `f` prend toujours **deux arguments** :

- `acc` : l'**accumulateur**, désigne le résultat cumulé jusqu'ici (initialisé avec le premier élément) ;

- `x` : l'élément courant de la liste.

```
from functools import reduce
nombres = [1, 2, 3, 4, 5]
produit = reduce(lambda acc, x: acc * x, nombres) # produit désigne 120
```

- Étape 1: `acc = 1`, `x = 2` → $1 \times 2 = 2$ est le nouveau résultat intermédiaire passé *via* `acc`.
- Étape 2: `acc = 2`, `x = 3` → $2 \times 3 = 6$ est le nouveau résultat intermédiaire passé *via* `acc`.
- Étape 3: `acc = 6`, `x = 4` → $6 \times 4 = 24$ est le nouveau résultat intermédiaire passé *via* `acc`.
- Étape 4: `acc = 24`, `x = 5` → $24 \times 5 = 120$ est la valeur finale : `produit` désigne 120.

Équivalent récursif explicite

```
def produit_rec(lst):
    if len(lst) == 1:
        return lst[0]
    return lst[0] * produit_rec(lst[1:])

produit_rec(nombres) # renvoie 120
```

L'équivalent récursif rend visible ce que `reduce` fait implicitement : on traite le premier élément, puis on rappelle la fonction sur le reste — jusqu'à ce qu'il n'en reste qu'un.

En NSI

`reduce` est hors programme, mais elle illustre parfaitement l'idée fonctionnelle : pas de variable d'accumulation visible, pas d'effet de bord.

L'équivalent impératif serait un `total = 1` suivi d'une boucle `for` multipliant `total` par chaque élément.

3. Pourquoi le paradigme fonctionnel ?

- **Prévisibilité** : une fonction pure est facile à tester — il suffit de vérifier ses entrées/sorties.
- **Absence de *bugs* liés à l'état partagé** : sans variables globales mutables, de nombreux bugs de concurrence disparaissent.
- **Parallélisation** : des fonctions pures peuvent s'exécuter en parallèle sans risque d'interférence.

En pratique

Python n'est pas un langage *purement* fonctionnel (comme Haskell ou OCaml), mais il permet d'adopter un *style* fonctionnel quand cela simplifie le code.

Aux écrits du baccalauréat NSI, la notion apparaît surtout à travers l'usage des compréhensions de listes et des fonctions récursives (sans effet de bord).

Si la notion devait être un jour rencontrée plus en profondeur dans un sujet, l'essentiel serait certainement de savoir faire la distinction entre fonction *pure* et fonction *avec effets de bord*.